Building Intelligent Workflows with LangChain: A Practical Exploration

Exploring LangChain Concepts with a Free LLM

In this notebook, I explore how LangChain helps transform simple language models into structured and intelligent systems.

Rather than treating AI as a black box, I focus on breaking it down step by step — starting from basic model interaction to building reusable workflows.

To avoid dependency on paid APIs, I use an open-source model, which allows me to focus purely on understanding the core concepts of prompt design, chaining, and workflow creation.

```
!pip install -q langchain transformers torch
```

Importing Required Libraries

LangChain provides modular tools, and I am combining it with a transformer-based model.

```
from transformers import pipeline
from langchain_core.prompts import PromptTemplate
```

Initializing the Language Model

Here, I am using a lightweight open-source model (GPT-2) to simulate LLM behavior.

```
llm = pipeline("text-generation", model="gpt2")
```

```
Loading weights: 100%  148/148 [00:00<00:00, 436.64it/s, Materializing param=transformer.wte.weig

GPT2LMHeadModel LOAD REPORT from: gpt2
Key                   | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; r
```

Step 1: Basic LLM Interaction

Let me first observe how the model responds without any structured prompt.

This helps in understanding how the model behaves without any structured input.

```
response = llm("Explain Artificial Intelligence in one simple sentence
print(response[0]['generated_text'])
```

```
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_n
Explain Artificial Intelligence in one simple sentence and let it learn

If you're not in the mood for math, this is very helpful for learning. It will m

Don't want to know anything about Artificial Intelligence? Just read this book c

A second tool is Computer Vision.

A third tool is Cognitive Learning.

A fourth tool is Artificial Intelligence.

And finally, a fifth tool is Artificial Intelligence for Business.

If you want to learn more about all of these things, there are lots more resourc

The Science of AI
```

Step 2: Creating a Prompt Template

Instead of writing prompts manually every time, I can design reusable templates.

```
prompt = PromptTemplate.from_template(
    "Explain {topic} in a simple and beginner-friendly way"
)
```

```
formatted_prompt = prompt.format(topic="Machine Learning")
print(formatted_prompt)
```

```
Explain Machine Learning in a simple and beginner-friendly way
```

Step 3: Building a Simple Pipeline

Here I combine the prompt template with the language model.

This step connects the prompt and model, forming a basic workflow similar to LangChain pipelines.

```
final_prompt = prompt.format(topic="Neural Networks")

response = llm(final_prompt, max_length=60)
print(response[0]['generated_text'])
```

```
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=60) seem to have been set. `max_n
Explain Neural Networks in a simple and beginner-friendly way.

As a general rule, the most important part of this tutorial is to get your brain

How to learn Python Neural Networks

If you haven't already done this tutorial, then you might want to skip it and re

The first part of this tutorial is all about learning basic concepts. The rest o

The second part of the tutorial is about developing neural networks. The other p

Don't forget to check the sections on learning by using the Python Neural Networ

If you don't know the basics in this tutorial and don't want to know the basic c

Learn how Python Neural Networks work by learning
```

Note: In modern LangChain, pipelines are created using a chaining mechanism where prompts and models are combined. Here, I simulate the same idea manually using a prompt template and a language model.

Step 4: Creating a Reusable Workflow

Now I extend the same pipeline to generate real-world use cases.

```
usecase_prompt = PromptTemplate.from_template(
    "Give 3 real-world applications of {topic}"
)

text = usecase_prompt.format(topic="Artificial Intelligence")

response = llm(text, max_length=80)
print(response[0]['generated_text'])
```

```
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set. `max_n
Give 3 real-world applications of Artificial Intelligence, including AI and real

Learn how you can create real-world applications from real-world problems.

Get a personalized, personalized experience with The Future Of Life

Build personalized, personalized apps that help you find life the way you want t

We believe that our products will be the most meaningful tool for people to use.

It's time to take a step back and think of a real-world application.

Here are some of the major innovations from The Future Of Life team:

Recharge your batteries with one of The Future Of Life's rechargeable rechargeab

Make the smart phone your home in seconds.

Create a smart, personalized website for a business, online store, or social net

Take your company to the next level by offering a unique experience, making it e

Create and discover new ideas and solutions.

Get to know your customers and your customers-to-be.

Our partners at The Future Of Life can help you find your perfect customer.

To learn more about The Future Of Life's products, visit TheFutureOfLife
```

Step 5: Testing with Multiple Inputs

To make the system dynamic, I test multiple topics.

```
topics = ["Data Science", "Robotics", "Cyber Security"]

# Updated prompt (to avoid repetition and improve quality)
prompt = PromptTemplate.from_template(
    "Explain {topic} in a simple, clear and unique way with one real-w
)
```

```
for topic in topics:
    text = prompt.format(topic=topic)

    result = llm(
        text,
        max_length=80,
        num_return_sequences=1,
        do_sample=True,
        temperature=0.7,
        pad_token_id=50256,
        truncation=True
    )

    print(f"\n🔹 Topic: {topic}")
    print(result[0]['generated_text'])
```

```
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set. `max_n
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set. `max_n

🔹 Topic: Data Science
Explain Data Science in a simple, clear and unique way with one real-world examp

About the Author

David F. Feltman is a CTO at Cauldron, an industry leader in the field of HTML,
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set. `max_n

🔹 Topic: Robotics
Explain Robotics in a simple, clear and unique way with one real-world example -

This video will explain all the major basic concepts of Robotics in a simple, cl

This video will explain all the major basic concepts of Robotics in a simple, cl

🔹 Topic: Cyber Security
Explain Cyber Security in a simple, clear and unique way with one real-world exa

One of the things that keeps you busy while you're there is the fact that there

So go out and meet another new person and join them and see for yourself what a

You want to know if they are right for you?

This is the moment.

Maybe they have some great ideas. Maybe they are good at it.

But you don't know if it will work out?

What can you do?

You could simply follow them on Twitter or Facebook or wherever.

You could do that every day, every day...
```

```
All right, I don't know what I would do with all of this time, but I could use i

I have a friend who is so focused on his computer that he has been doing all of

He's actually been going to his computer for 15 years now, and he's not a comput
```

From these experiments, I observed:

- The structure of a prompt significantly influences the quality of output
- Reusable templates make workflows more efficient and scalable
- Even simple models can generate meaningful responses when guided properly
- Breaking the system into steps improves both understanding and control
- Open-source models like GPT-2 sometimes produce repetitive or less accurate outputs compared to advanced LLMs

These observations highlight how LangChain focuses not just on models, but on designing structured AI workflows.

Final Reflection

Through this notebook, I understood that:

• LangChain is more about **how we structure AI workflows**

• Prompt engineering plays a key role

• Pipelines help in building scalable systems

Even without paid APIs, the core ideas of LangChain can be clearly demonstrated.

This experiment helped me understand that building AI systems is not just about using powerful models, but about designing the right workflow around them.

What Makes This Approach Unique?

◆ API-Based Approach

- Uses API-based models
- Focus on direct understanding
- Faster results but API dependent

- My Approach:
  - Uses open-source model (no API dependency)
  - Focus on concept understanding
  - Demonstrates real workflow design
  - Easy to reproduce and extend